

Imbalanced Data in Machine Learning

Mastering Evaluation, Sampling & Real-World ML Workflows

Presented by **Nancy Ahmed**

Machine Learning



Why Accuracy Lies



SMOTE & Variants



Hybrid Methods



Leakage Prevention



Real-World Demo

Today's Agenda

01 Fundamentals

- Imbalance Problem
- Why Accuracy Fails
- Evaluation Metrics
- Confusion Matrix

02 Sampling Methods

- Oversampling & SMOTE
- SMOTE Variants
- Undersampling
- Hybrid Methods

03 Practical Workflow

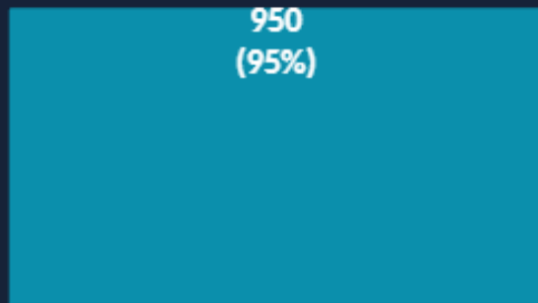
- Train-Test Split Leakage
- Class Weights
- Pipelines
- Real-World Demo

What is Imbalanced Data?



A dataset where one class appears much more frequently than another — making it difficult for models to learn the minority class.

Class Distribution



Normal (Class 0)



Fraud (Class 1)

Real-World Examples



Credit Card Fraud



Disease Detection



Network Intrusion



Defect Detection



Rare Event Prediction



The Accuracy Paradox

95%

Accuracy

Predicts ONLY Class 0

Confusion Matrix

	Pred 0	Pred 1
Actual 0	950 ✓	0
Actual 1	50 ✗	0

Recall

= 0

*Completely useless
for fraud detection*



Stop using Accuracy as your primary metric on imbalanced datasets.

A model predicting ONLY the majority class can achieve 95% accuracy while completely failing its core task.

Critical Evaluation Metrics



Precision

$$TP / (TP + FP)$$

When the model predicts positive, how often is it correct?

★ *Low false alarm rate*



Recall

$$TP / (TP + FN)$$

How many actual positives were correctly found?

★ *Critical for fraud & medical*



F1 Score

$$2 \times (P \times R) / (P + R)$$

Harmonic mean of Precision and Recall.



ROC-AUC

Area Under ROC Curve

Threshold-independent evaluation of classifier quality.



PR Curve (Precision-Recall Curve) is often MORE informative than ROC-AUC on severely imbalanced datasets.

Confusion Matrix — Deep Dive

	Predicted 0	Predicted 1
Actual 0 ↓	TN True Negative <i>Correctly predicted NEGATIVE</i>	FP False Positive <i>Predicted positive — was negative (Type I)</i>
Actual 1	FN False Negative <i>Predicted negative — was positive (Type II)</i>	TP True Positive <i>Correctly predicted POSITIVE</i>

Quick Formulas

Accuracy

$$(TP+TN)/Total$$

Precision

$$TP/(TP+FP)$$

Recall

$$TP/(TP+FN)$$

F1

$$2 \cdot P \cdot R / (P + R)$$

Specificity

$$TN/(TN+FP)$$

Solutions Roadmap

1

Oversampling



Increase minority class samples

2

Undersampling



Reduce majority class samples

3

Hybrid Methods



Combine over + undersampling

4

Class Weights



Penalize minority errors more

5

Threshold Tuning



Adjust decision boundary

6

Ensemble Methods



BalancedRF,
EasyEnsemble,
RUSBoost

7

Anomaly Detection



One-class classifiers

Random Oversampling

How It Works

- 1 Take existing minority class samples
- 2 Randomly duplicate them until classes are balanced
- 3 No new information is added — copies are exact
- 4 Simple and fast to implement

```
from imblearn.over_sampling import RandomOverSampler  
ros = RandomOverSampler()  
X_res, y_res = ros.fit_resample(X, y)
```



Advantages

- Simple and easy to use
- Preserves all minority information



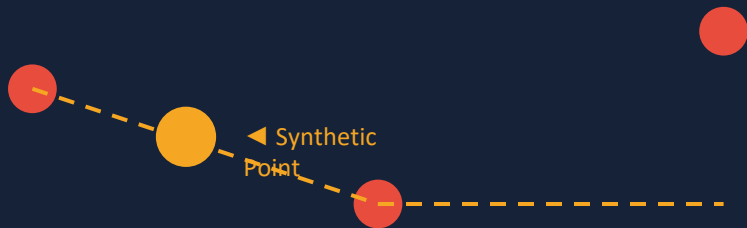
Disadvantages

- High overfitting risk
- Duplicates exact samples — no diversity
- Model memorizes rather than generalizes

SMOTE — The Heart of Oversampling

Synthetic Minority Over-sampling Technique

Intuition: Creating Synthetic Points



Instead of copying: generate new points between k-nearest neighbors

```
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_res, y_res = smote.fit_resample(X, y)
```



Advantages

- Less overfitting than duplication
- More generalized decision boundary



Disadvantages

- May create noisy synthetic samples
- Can produce unrealistic combinations
- Struggles near class overlap

SMOTE Variants

Borderline-SMOTE

Generates samples only near the decision boundary — where it matters most

★ *Best For: Overlapping classes*

ADASYN

Adaptive Synthetic — generates MORE samples for harder, mis-classified regions

★ *Best For: Difficult imbalanced datasets*

KMeans-SMOTE

Clusters minority data first, then applies SMOTE inside each cluster intelligently

★ *Best For: Complex data distributions*

SVMSMOTE

Uses SVM support vectors to identify critical boundary regions for synthesis

★ *Best For: Nonlinear decision boundaries*

SMOTENC

Handles mixed datasets with both categorical and numerical features

★ *Best For: Real-world tabular data*

SMOTEN

Handles purely categorical datasets using value frequency for synthesis

★ *Best For: All-categorical features*

Undersampling Methods

Random Undersampling

Remove majority class samples randomly until classes are balanced.

✓ Fast, reduces training time ✗ Risks losing important patterns

```
from imblearn.under_sampling import RandomUnderSampler
```

Tomek Links

Identify pairs of majority-minority samples that are nearest neighbors and remove the majority ones — cleaning the boundary.

✓ Cleaner class boundaries ✗ Conservative — may not balance fully

```
from imblearn.under_sampling import TomekLinks
```

NearMiss

Keep only majority samples closest to minority class (versions 1, 2, 3 vary in distance logic).

✓ Preserves boundary-relevant data ✗ Can keep redundant samples

```
from imblearn.under_sampling import NearMiss
```

Edited Nearest Neighbours (ENN)

Remove noisy or misclassified samples that are inconsistent with their neighborhood.

✓ Acts as a dataset cleaner ✗ Minimal reduction in majority

```
from imblearn.under_sampling import EditedNearestNeighbours
```

Hybrid Methods

Combining oversampling + undersampling is a very common and powerful professional practice.

SMOTE + Tomek Links → SMOTETomek

```
from imblearn.combine import  
SMOTETomek
```

Original
Dataset



SMOTE adds
minority samples



Tomek removes
noise/ambiguous



Clean & Balanced
Dataset ✓

SMOTE + Edited Nearest Neighbours → SMOTEENN

```
from imblearn.combine import SMOTEENN
```

Original
Dataset



SMOTE adds
synthetic samples



ENN removes
misclassified



Cleaner &
Balanced ✓



Professional tip: Hybrid methods are the industry standard for high-stakes imbalanced classification tasks.



HUGE Common Mistake

Data Leakage

✗ WRONG Workflow

```
# Apply SMOTE to all data FIRST
X_res, y_res = smote.fit_resample(X, y)

# Then split — WRONG!
X_train, X_test, y_train, y_test = train_test_split(X_res, y_res)
```



✓ CORRECT Workflow

```
# Split FIRST — test set stays untouched
X_train, X_test, y_train, y_test = train_test_split(
    X, y, stratify=y, random_state=42)

# Then apply SMOTE only on train
X_tr, y_tr = smote.fit_resample(X_train, y_train)
```

Correct ML Workflow:

Train-Test
Split



Apply SMOTE
on Train Only



Train
Model



Evaluate on
Untouched Test

Stratification & Class Weights

`stratify=y` — ALWAYS Use This

Without stratification, the minority class may disappear entirely from the test set, making evaluation meaningless.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,      # ← Always include!
    random_state=42
)
```



Class Weights

Instead of resampling, make minority mistakes more expensive in the loss function. No synthetic data needed.

```
LogisticRegression(
    class_weight='balanced')

RandomForestClassifier(
    class_weight='balanced')
```

Model Selection & Threshold Tuning



Tree-Based Ensembles Handle Imbalance Better

✓ Random Forest

✓ XGBoost

✓ LightGBM

✓ CatBoost

Ensemble Methods for Imbalance

Balanced Random Forest • EasyEnsemble • RUSBoost

Sensitive to Imbalance — Need Extra Care

Logistic Regression • SVM • KNN
→ Require scaling + resampling + class weights



Threshold Tuning

Default threshold = 0.5

Lowering the threshold increases Recall (finds more positives).

Raising it increases Precision (fewer false alarms).

★ Critical for: Fraud detection, Medical diagnosis

Common Mistakes

1

Using accuracy as the primary metric on imbalanced data

[Critical]

2

Applying SMOTE BEFORE train-test split (Data Leakage)

[Critical]

3

Forgetting stratify=y in train-test split

[High]

4

Oversampling the test data — test must stay pristine

[Critical]

5

Evaluating model on the resampled test set

[High]

6

Ignoring the precision-recall tradeoff for the use case

[Medium]

Method Comparison Table

Method	Main Idea	Advantages	Disadvantages	Best For
Random Oversampling	Duplicate minority samples	Simple, fast	Overfitting risk	Quick baseline
SMOTE	Synthetic interpolation between neighbors	Better generalization	Synthetic noise	General use
Random Undersampling	Remove majority samples randomly	Fast training	Information loss	Large datasets
Tomek Links	Remove boundary-overlapping majority samples	Cleaner boundaries	Minor reduction	Boundary cleanup
Class Weights	Penalize minority errors in loss	No synthetic data	Model-dependent	Clean approach
SMOTETomek / SMOTEENN	Oversample + clean boundaries	Best of both worlds	Computationally heavier	Production ML

Key Takeaways



Accuracy is deceptive — always use F1, Recall, AUC on imbalanced data



SMOTE creates synthetic minority points — far better than simple duplication



Always split FIRST, then apply SMOTE only on training data — never leak



Use stratify=y in every train-test split — preserve class proportions



Class weights offer a clean, no-synthetic alternative built into most models

"A model with 99% accuracy can still be completely useless on imbalanced data."



Thank You!